IST604 – Web Services & Distributed Computing

Chapter 2: Introduction to Web Services — Lecture Summary

2.1 What is a Web Service?

A **web service** is a standards-based, language-agnostic software entity that enables two different applications — potentially written in different programming languages and running on different platforms — to communicate and exchange data over a network using standardized protocols like HTTP.

Key Characteristics

- **Standards-based** — built on universally accepted specifications
- **Vendor & transport neutral** — not tied to any particular vendor or protocol
- **Language-agnostic** — Java, .NET, PHP, and others can all interoperate
- **Formatted requests / application-specific responses** — structured input yields structured output
- **Operates on remote machines** — communication happens across a network

Web Service vs. API

- All web services are APIs, but **not all APIs are web services**
- A web service **must** operate over a network; an API can be local (e.g., an OS library)

2.2 Core Functionality

Feature	Description
Interoperability	Bridges applications across different languages and platforms
Standardized Messaging	Data exchanged using XML or JSON
Loose Coupling	Client and server are independent; internal changes on one side don't break the other

---

2.3 Benefits of Web Services

- **Loose Coupling** — promotes independence between systems
- **Ease of Integration** — acts as "glue" between data silos within and across organizations
- **Service Reuse** — a function is coded once and reused by multiple consuming applications

---

2.4 Architecture of Web Services

2.4.1 Simplest Model

Two components:

- **Service Provider** — presents the interface and implementation
- **Service Consumer (Requester)** — uses the web service

2.4.2 Service-Oriented Architecture (SOA)

SOA is an **architectural style** that designs software as a collection of independent, reusable services focused on business functionality.

Three core SOA roles:

Role	Responsibility
Service Provider	Develops, deploys, and publishes service interfaces to a registry
Service Broker/Registry	A directory (using UDDI) where services are registered and discovered
Service Requester/Consumer	Finds services via the registry, binds to the provider, and invokes them

SOA vs. Web Services:

- SOA is the "*what*" (architectural blueprint)
- Web services are the "*how*" (technical implementation)

2.4.3 Communication Models

RPC-Based (Synchronous)

- Client sends a request and **waits** for a response before continuing
- **Tightly coupled** — implemented with remote objects
- Examples: CORBA, RMI

Messaging-Based (Asynchronous/Synchronous)

- **Loosely coupled**, document-driven
- Client sends an entire document and does **not wait** for a response
- The provider may or may not return a message

2.4.4 Web Services Standards

Primary Standards

Standard	Description
SOAP	XML-based messaging protocol; W3C-accepted; transported over HTTP, SMTP, FTP
WSDL	XML-based language describing a web service's interface, methods, and endpoint
UDDI	XML-based framework for registering, discovering, and integrating web services
REST	Lightweight architectural style using HTTP methods (GET, POST, PUT, DELETE) + JSON/XML
XML	W3C standard markup language for structuring and exchanging data

Standard	Description
JSON	Lightweight, text-based, key-value data format; a leaner alternative to XML

Other Standards

- **HTTP 1.1/2** — foundation of web communication
- **OpenAPI/Swagger** — standard for describing RESTful APIs
- **WS-Security** — message-level security for SOAP
- **WS-Addressing** — transport-neutral message addressing
- **WS-ReliableMessaging** — guaranteed delivery in distributed systems
- **MTOM** — optimizes SOAP message transmission (binary data)

2.5 SOAP (Simple Object Access Protocol)

SOAP is the messaging protocol between the service provider and requester.

SOAP Message Structure

A SOAP message is a standard XML document with:

- **Envelope** (root) — identifies the document as SOAP and contains all elements
- **Header** (optional) — metadata (e.g., authentication, transaction info)
- **Body** (required) — actual request/response payload
- **Fault** (optional, inside Body) — describes errors and exceptions
- **Attachments** — binary data in MIME encoding

SOAP Exchange Flow

1. Client constructs a SOAP request and sends it over HTTP
2. Message contains an Envelope with Header + Body
3. Server validates, processes, and executes the required logic
4. Server returns a SOAP response using the same protocol
5. Response Envelope contains the payload
6. Client receives and processes the response

2.6 SOAP vs. REST

Feature	SOAP	REST
Type	Protocol	Architectural Style
Data Format	XML only	JSON, XML, HTML, Text
Transport	HTTP, SMTP, JMS, TCP	HTTP/HTTPS only
State	Stateful or stateless	Inherently stateless
Security	WS-Security (message-level)	HTTPS, OAuth 2.0, JWT

Feature	SOAP	REST
Performance	Slower (verbose XML)	Faster (lightweight JSON)
Contract	Required (WSDL)	Optional (OpenAPI)
Caching	Not cacheable	HTTP caching supported
Error Handling	Fault element in XML	HTTP status codes (404, 500)

When to Use Each

- **REST** → Modern web/mobile apps, public APIs, microservices (speed & scalability priority)
- **SOAP** → Legacy system integrations, banking/healthcare, enterprise systems requiring strict security and ACID compliance

---

2.7 WSDL (Web Services Description Language)

WSDL is an XML-based language that acts as a **formal interface contract** between client and server for SOAP services — defining the *"how,"* *"what,"* *and* *"where"* of a web service.

WSDL Document Structure

Element	Purpose
`<definitions>`	Root element; declares namespaces
`<types>`	Defines data types using XML Schema (XSD)
`<message>`	Defines data units being communicated (like function parameters)
`<portType>` / `<interface>`	Most critical — groups abstract operations and their input/output messages
`<binding>`	Links portType to a concrete protocol (e.g., SOAP over HTTP)
`<service>`	Groups related endpoints
`<port>` / `<endpoint>`	Specifies the actual network address (URL)

WSDL Operational Patterns

- **One-way** — service receives a message, no response
- **Request-Response** — most common; service receives and returns a response
- **Solicit-Response** — service initiates and waits for a response
- **Notification** — service sends a message without expecting a response

WSDL Development Approaches

- **Top-Down (Contract-First)** — write the WSDL first, then generate code (e.g., `wsimport`, `WSDL2Java`)
- **Bottom-Up (Code-First)** — write code first, then generate WSDL (e.g., `wsgen`, `Java2WSDL`)

WSDL Version Comparison

Feature	WSDL 1.1	WSDL 2.0
Status	W3C Note (widely used)	W3C Recommendation
Root element	`<definitions>`	`<description>`
Interface element	`<portType>`	`<interface>`
Endpoint element	`<port>`	`<endpoint>`
Inheritance	Not supported	Supported

2.8 Implementing Web Services — Key Steps

1. **Requirements & Design** — define functionality, select REST or SOAP, establish a contract (WSDL for SOAP)
2. **Development** — write business logic, expose operations, handle data type mapping (XML/JSON)
3. **Deployment** — configure an application server (WebLogic, GlassFish), publish the service
4. **Testing & Validation** — use SoapUI or Postman; verify security (Auth, SSL)
5. **Discovery & Consumption** — register in UDDI (optional); generate client proxies from WSDL

2.9 Java Web Services Development

Java provides two primary APIs for web service development:

API	Purpose	Implementations
JAX-WS	SOAP-based web services	RPC Style, Document Style
JAX-RS	RESTful web services	Jersey, RESTeasy

- JAX-WS is available from **Java 6+** (core libraries included)
- **Spring Boot** is the modern industry standard but is not covered in this course